Power-Performance-Area Trade-offs in FPGA-Based FIR Accelerators: A Comprehensive Architectural Analysis

A Thesis Submitted

In Partial Fulfilment of the Requirements for the Degree of

Bachelor of Technology

In

Electronics and Communication Engineering

Submitted by

Saptarshi Pal & Gopesh Saha

22EC8021 & 22EC8086

Under the Supervision of

Dr. Rajashree Das

Assistant Professor

ELECTRONICS AND COMMUNICATION ENGINEERING DEPARTMENT

National Institute of Technology Durgapur

West Bengal – 713209

India

May, 2026

Department of Electronics and Communication

National Institute of Technology Durgapur

Mahatma Gandhi Avenue, Durgapur-713209

West Bengal

India

BONAFIDE CERTIFICATE

This is to certify that **Saptarshi Pal & Gopesh Saha** bearing Roll No. **22EC8021 & 22EC8086** have successfully completed their project on the thesis entitled, **"Power-Performance-Area Trade-offs in FPGA-Based FIR Accelerators: A Comprehensive Architectural Analysis "** under our supervision for submission to National Institute of Technology Durgapur towards partial fulfilment for the award of the Degree of Bachelor of Technology in Electronics and Communication Engineering and this thesis is an authentic record of their own work carried out under our supervision.

.....

Dr. Rajashree Das
Assistant Professor,
Electronics and Communication Engineering Department,
National Institute of Technology Durgapur

Department of Electronics and Communication

National Institute of Technology Durgapur

Mahatma Gandhi Avenue, Durgapur-713209

West Bengal

India

CERTIFICATE OF APPROVAL

The foregoing thesis entitled **"Power-Performance-Area Trade-offs in FPGA-Based FIR Accelerators: A Comprehensive Architectural Analysis"** is hereby approved as a study of a technology subject carried out in a manner satisfactory to warrant its acceptance as a prerequisite to the degree for which it has been submitted. It is understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn thereon. The thesis is approved only for the purpose for which it is submitted.

……

Examiner(s)

Department of Electronics and Communication

National Institute of Technology Durgapur

Mahatma Gandhi Avenue, Durgapur-713209

West Bengal

India

DECLARATION

We, **Saptarshi Pal & Gopesh Saha** bearing Roll No. **22EC8021 & 22EC8086** of B.Tech in Electronics and Communication Engineering hereby declare that our B. Tech. Project / Thesis titled **"Power-Performance-Area Trade-offs in FPGA-Based FIR Accelerators: A Comprehensive Architectural Analysis"** has been carried out independently by us. We also declare that we have not indulged in any form of plagiarism to carry out the project and while writing this report. The work or ideas of other authors which are utilized in this report has been properly acknowledged and mentioned in the Bibliography. The work presented in this dissertation or any part thereof has not been submitted anywhere else for the award of any degree in another university. We undertake total responsibility if any traces of plagiarism are found at any later stage. In such a case our degree will be cancelled automatically.

Date: May, 2025

Place: NIT, Durgapur

Saptarshi Pal Roll Number: 22EC8021	Gopesh Saha Roll Number: 22EC8086

ACKNOWLEDGEMENTS

We would like to express our profound gratitude to our project guide **Dr. Rajashree Das**, for her unwavering support, invaluable guidance, and constant encouragement throughout this research. Her expertise and insightful feedback have been instrumental in shaping this work and enabling us to overcome various challenges.

We are also deeply grateful to the faculty and staff of the Department of Electronics and Communication Engineering (ECE) at NIT Durgapur, for providing a resource-rich environment that greatly facilitated our research.

We extend our heartfelt appreciation to our parents and friends for their unwavering support, patience, and constant motivation throughout this journey. Their belief in us have been an endless source of strength.

Finally, we would like to thank all those who contributed, directly or indirectly, to the successful completion of this thesis.

Date: May, 2026

Place: Durgapur

……

(SAPTARSHI PAL)
22EC8021

……

(GOPESH SAHA)
22EC8086

ABSTRACT

The deployment of computationally intensive workloads in edge computing and high-speed communication systems is severely bottlenecked by the reliance on hardware multipliers. On Field Programmable Gate Arrays (FPGAs), conventional Multiply-Accumulate (MAC) arrays rapidly exhaust dedicated Digital Signal Processing (DSP) slices, introduce deep critical path delays, and consume prohibitive amounts of dynamic power. This thesis presents a comprehensive Design Space Exploration (DSE) of multiplier-free datapath architectures to systematically optimize the Power-Performance-Area (PPA) envelope. Six distinct hardware models, spanning conventional pipelined MACs to Canonical Signed Digit (CSD) shift-and-add structures, were designed at the Register Transfer Level (RTL) and post-route verified on the AMD Xilinx Artix-7 FPGA.

As a primary contribution, this work provides the FPGA post-routing validation of an advanced Approximate Distributed Arithmetic (DA) architecture. By unifying input truncation, Radix-8 Booth Encoding, and an Approximate Wallace Tree constructed from Lower-part OR Adders (LOA), the carry-propagation chains in the Least Significant Bits (LSBs) are structurally eliminated. Post-implementation results demonstrate that this approximate architecture consumes zero DSP slices and requires only 0.078% of available Look-Up Tables (LUTs).

Furthermore, a throughput-aware evaluation exposes the performance illusion of conventional time-multiplexed MAC designs: while a folded MAC achieves a 207.9 MHz raw frequency, its multi-cycle nature throttles effective throughput to just 25.99 Mega-Samples Per Second (MSPS). In contrast, the fully parallel Approximate DA datapath achieves a maximum operating frequency of 166.0 MHz and delivers a true single-cycle throughput of 166.0 MSPS, while minimizing dynamic logic power to under 1 milliwatt. Ultimately, this research proves that strategically trading exact mathematical precision for approximate logic yields highly scalable, energy-efficient, and DSP-free datapath accelerators ideal for resource-constrained edge environments.

NOMENCLATURE

Abbreviation	Full form
ASIC	Application-specific Integrated Circuit
CSD	Canonical Signed Digit
DA	Distributed Arithmetic
DSE	Design Space Exploration
DSP	Digital Signal Processing/Processor
FF	Flip-flop
FIR	Finite Impulse Response
FPGA	Field Programmable Gate Array
HDL	Hardware Description Language
IP	Intellectual Property
LMS	Least Mean Squares
LOA	Lower-part OR Adder
LSB	Least Significant Bit
LUT	Look-up Table
MAC	Multiply-Accumulate
MSB	Most Significant Bit
MSE	Mean Squared Error
PPA	Power, Performance and Area
RTL	Register Transfer Level
SoC	System on Chip
WNS	Worst Negative Slack

SYMBOLS

Symbol	Description
N	Number of filter taps / order of the datapath
x[n]	Discrete-time input data sample
y[n]	Discrete-time output data sample
b[k]	Quantized fixed-point filter coefficient
F_{max}	Maximum operating clock frequency (MHz)
W	Word length (bit-width) of the hardware register

LIST OF FIGURES

LIST OF TABLES

CONTENTS

CHAPTER 1: INTRODUCTION

1.1 The Hardware Multiplier Problem in Edge Computing

The relentless proliferation of data-intensive workloads in embedded and edge computing systems has placed unprecedented demands on hardware accelerator design. Applications ranging from real-time digital filtering in communication front-ends to inference engines in edge AI deployments all share a common architectural bottleneck: the hardware multiplier. In these systems, the Multiply-Accumulate (MAC) operation is the fundamental computational primitive, and its implementation in silicon directly dictates the Power-Performance-Area (PPA) envelope of the entire accelerator.

On Field Programmable Gate Arrays (FPGAs), hardware multiplication is a resource-critical operation handled by dedicated, hard-silicon Digital Signal Processor (DSP) slices. While DSP slices deliver high-throughput arithmetic at low logic overhead, they are a severely constrained resource. The AMD Xilinx Artix-7 XC7A200T, the target device of this research, provides only 740 DSP slices against 134,600 general-purpose Look-Up Tables (LUTs). In a parallel datapath architecture, where each filter tap demands a dedicated multiplier, this pool of DSP resources is exhausted rapidly. When the synthesis tool is forced to construct multipliers from LUTs instead, the result is severe routing congestion, exponentially degraded timing, and unsustainable dynamic power consumption.

This thesis confronts this problem directly. The Finite Impulse Response (FIR) filter, a canonical MAC-heavy datapath used ubiquitously in signal processing and edge AI preprocessing pipelines, is selected as the evaluation benchmark. From a digital hardware perspective, an FIR filter is simply a tapped delay line feeding a parallel array of multipliers whose outputs are summed by an adder tree. This structure stress-tests every dimension of the FPGA resource budget simultaneously, DSP slices, LUT logic, flip-flop registers, and routing fabric, making it an ideal and rigorous proxy for evaluating architectural optimization strategies.

1.2 Limitations of Conventional Optimization Strategies

The standard VLSI response to multiplier-induced timing violations is register pipelining: inserting flip-flop barriers between arithmetic stages to sever the critical path and boost the maximum operating frequency (F_max). While effective at achieving timing closure, pipelining introduces a fundamental and often underappreciated throughput paradox when applied naively to MAC-folded architectures.

A time-multiplexed MAC architecture, which reuses a single multiplier across all filter taps sequentially, achieves a deceptively high raw clock frequency by minimizing combinational logic depth. However, for an N-tap FIR filter, it requires N clock cycles to produce a single output sample, directly throttling the effective data throughput to F_max / N. For the 8-tap design evaluated in this work, a MAC architecture operating at 207.9 MHz yields an effective throughput of only 34.65 Mega-Samples Per Second (MSPS), inferior to architectures with substantially lower raw clock speeds but fully parallel, single-cycle datapaths.

Beyond the MAC throughput illusion, standard pipelining of a parallel datapath aggressively consumes DSP slices: implementation results demonstrate that the pipelined architecture seizes 1.081% of the available DSP resources on the target device, compared to just 0.135% for the unpipelined baseline. This 8x escalation in DSP consumption is unsustainable in real-world deployments where the FPGA fabric must simultaneously host control logic, communication interfaces, and multiple accelerator cores.

1.3 The Approximate Computing Paradigm

A compelling alternative emerges from the observation that a large class of signal processing and machine learning workloads are inherently error-tolerant. FIR filtering, digital image processing, and neural network

inference all operate on noisy or quantized data streams; the end-to-end system accuracy degrades gracefully when a small, bounded error is introduced at the hardware arithmetic level. This observation motivates the paradigm of Approximate Computing: the deliberate, engineered relaxation of mathematical exactness in exchange for measurable gains in silicon area, power efficiency, and operating frequency.

In the context of multi-operand adder trees, the primary source of both critical path delay and dynamic switching power is the carry-propagate chain. Each carry bit must ripple sequentially from the Least Significant Bit (LSB) to the Most Significant Bit (MSB), creating a logic depth proportional to the operand word width. Critically, in most signal processing applications, the LSBs contribute the least to the output's dynamic range and perceptual quality. This asymmetry is the key insight exploited in this work.

By replacing exact full-adders in the lower bit positions of the Wallace tree with Lower-part OR Adders (LOA), a lightweight approximate arithmetic structure proposed in the IEEE TCAS-I literature, the carry chain is structurally eliminated from the LSB portion of the datapath. The resulting approximate adder evaluates the lower bits via static OR logic (a single gate level, zero carry propagation) while maintaining exact arithmetic in the MSB portion. The result is a datapath that is simultaneously smaller, faster, and more power-efficient, at the cost of a mathematically bounded, application-tolerable Mean Squared Error (MSE).

1.4 Proposed Architecture: Approximate Distributed Arithmetic

This research proposes and implements a novel Approximate Distributed Arithmetic (DA) architecture that unifies three advanced VLSI optimization techniques into a single, cohesive datapath:

1. **Input Truncation with Error Compensation:** The lower 4 LSBs of the 16-bit input data word are truncated prior to the multiply-accumulate stage, reducing the switching activity of the delay line and all downstream arithmetic. A constant bias term of $2^{(k-1)} = 8$ is added in hardware to compensate for the systematic truncation error, following the methodology established in the IEEE high-performance FIR filter literature.

2. **Radix-8 Booth Encoding:** Rather than generating one partial product per coefficient bit (standard binary multiplication), Radix-8 Booth encoding recodes each 8-bit coefficient into at most $\lceil 8/3 \rceil = 3$ signed partial products per tap. This reduces the height of the Wallace tree from 8 rows to 3 rows per tap slice, directly compressing the adder tree depth and the associated routing congestion.

3. **Approximate Wallace Tree with LOA Adders:** The three-row Booth partial product array is reduced using an Approximate Wallace Tree constructed from parametrized LOA sub-modules. Carry propagation is eliminated in the 3 LSB positions of each adder, fracturing the critical path while preserving the MSB accuracy that governs the filter's dynamic range.

The combination of these three techniques produces a datapath that requires zero DSP slices, demonstrates zero dependency on dedicated hardware multipliers, consumes under 1 milliwatt of dynamic logic power, and achieves a post-routing maximum operating frequency of 166.0 MHz, while delivering a fully parallel, single-cycle-per-sample throughput of 166.0 MSPS.

1.5 Objectives

To rigorously quantify the PPA trade-offs inherent to each optimization strategy, this research implements and evaluates six distinct hardware architectures on the AMD Xilinx Artix-7 XC7A200T FPGA. Each design is synthesized, placed, and routed under identical constraints, a 10 ns (100 MHz) target clock period on the same physical device, ensuring fair, implementation-validated comparison. The specific research objectives are:

1. Establish a post-routing PPA baseline using an unoptimized direct-form MAC architecture, and confirm its failure to achieve timing closure at 100 MHz.

2. Evaluate conventional optimization strategies, register pipelining and coefficient-symmetric folding, and quantify their impact on DSP consumption, F_max, and dynamic power.
3. Implement a multiplier-less Canonical Signed Digit (CSD) Shift-and-Add architecture to demonstrate the feasibility of zero-DSP operation using hardwired bit-shifts.
4. Implement and validate the proposed Approximate Distributed Arithmetic architecture, combining Radix-8 Booth encoding with an LOA-based Approximate Wallace Tree, as the primary contribution of this thesis.
5. Critically evaluate all six architectures across a unified PPA matrix, including LUT utilization, DSP slice consumption, post-routing Worst Negative Slack (WNS), derived F_max, effective throughput (MSPS), and total dynamic power, to identify the Pareto-optimal design point for edge-constrained FPGA deployment.

1.6 Contributions

The primary novel contributions of this thesis are as follows:

1. **First FPGA post-routing validation of Approximate Distributed Arithmetic:** The Approximate DA architecture of Jiang et al. (2018) was validated exclusively via ASIC synthesis at ST 28nm CMOS using Synopsys DC. This thesis provides the first complete post-placement-and-routing verification on a reconfigurable FPGA fabric (AMD Xilinx Artix-7 XC7A200T, -2 speed grade), where LUT mapping, carry-chain topology, and physical routing introduce fundamentally different timing and power characteristics compared to standard-cell synthesis.
2. **Unified six-architecture PPA comparison under identical implementation constraints:** A comprehensive Design Space Exploration spanning six architectural paradigms, Direct-Form MAC, Time-Multiplexed MAC, Pipelined, Coefficient-Symmetric, CSD Shift-and-Add, and Approximate DA, is conducted under a single, standardized Vivado implementation flow on the same physical device. This yields a directly comparable, implementation-validated PPA matrix that does not exist in prior FPGA-targeted literature.
3. **Throughput-aware evaluation metric exposing the MAC performance illusion:** This thesis formally introduces effective throughput (MSPS = F_max / Cycles-per-Output) as an architectural evaluation metric alongside raw F_max, demonstrating that a time-multiplexed MAC operating at 207.9 MHz delivers only 25.99 MSPS, inferior to the proposed Approximate DA architecture at 166.0 MSPS. This metric reframes the conventional raw-frequency standard used in prior FPGA design space explorations.
4. **Parametrized, synthesizable LOA sub-module for FPGA deployment:** A fully parametrized approxloaadder Verilog module supporting configurable word width and approximate bit-count is implemented and functionally verified. This module is directly reusable as a portable IP block in other FPGA datapath designs requiring approximate arithmetic.

1.7 Scope of the Present Work

The boundaries of this research are explicitly defined as follows to ensure reproducibility and focused evaluation.

Within scope:

- Six RTL architectures implemented in Verilog HDL and post-route verified on the AMD Xilinx Artix-7 XC7A200T (speed grade -2, package fbg676) using Xilinx Vivado
- Fixed-coefficient 8-tap FIR datapath with 16-bit signed input, 8-bit signed coefficients, and 32-bit signed accumulator output

- Uniform all-pass coefficient set ($h[k] = 0.125$, fixed-point representation 8'b00010000) used as a controlled benchmark to isolate the effect of architectural choices from coefficient-specific optimizations
- Post-implementation (post-routing) PPA metrics: LUT utilization, flip-flop count, DSP slice count, Worst Negative Slack (WNS), derived F_max, effective MSPS throughput, and total dynamic power, all extracted from Vivado implementation reports
- Functional RTL simulation and MSE-based output verification against a Python floating-point golden reference

Explicitly out of scope:

- ASIC synthesis, standard-cell mapping, physical layout, or parasitic extraction
- Floating-point or variable-precision arithmetic datapaths
- Adaptive coefficient update via the LMS algorithm, identified as a primary direction for future work in Chapter 5
- Cross-device portability evaluation beyond the Artix-7 family
- Formal mathematical error-bound proofs for the LOA approximation, the approximation error is empirically characterized via simulation MSE, not analytically bounded

1.8 Thesis Organization

The remainder of this thesis is structured as follows:

- **Chapter 2** surveys the evolution of hardware datapath architectures on FPGAs, from direct-form MAC implementations to multiplier-less CSD designs and approximate computing paradigms, grounding the proposed approach in the current state of the art.
- **Chapter 3** details the Register Transfer Level (RTL) design and Verilog HDL implementation of all six architectures, including the mathematical derivation of the quantization scheme, the Radix-8 Booth encoder truth table, the LOA sub-module parametrization, and the hardware-software co-design methodology used for functional verification.
- **Chapter 4** presents the post-implementation simulation waveforms, the complete PPA comparison matrix, and a quantitative analytical discussion of the results, culminating in the throughput-aware evaluation that exposes the MAC architecture's performance illusion.
- **Chapter 5** summarizes the research contributions, evaluates the hardware approximation trade-offs, and outlines three high-impact directions for future work: RISC-V coprocessor integration, LMS adaptive filtering, and ASIC synthesis for physical area characterization.

CHAPTER 2: LITERATURE REVIEW AND THEORETICAL BACKGROUND

2.1 The Direct-Form MAC Paradigm: Foundations and Bottlenecks

The Finite Impulse Response (FIR) filter is mathematically defined by the discrete-time convolution sum [1]:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

where x[n] is the discrete-time input signal, h[k] are the fixed filter coefficients, N is the number of filter taps, and y[n] is the filtered output. At the hardware level, each term h[k] · x[n-k] constitutes one Multiply-Accumulate (MAC) operation, making the FIR datapath a canonical MAC-array structure.

The earliest and most direct hardware realization is the Direct-Form MAC architecture, wherein a tapped delay line of N shift registers holds the most recent N input samples. Each register output drives a dedicated parallel hardware multiplier, and the N resulting products are reduced by a multi-operand binary adder tree to produce y[n]. The critical path, the longest combinational delay from input to registered output, traverses the multiplier and then the full adder tree depth. For an N-tap design, the adder tree has $\lceil \log_2 N \rceil$ levels; this depth grows logarithmically with N, directly degrading the maximum achievable clock frequency F_max[2].

2.2 Conventional Optimization: Pipelining and Coefficient Symmetry

2.2.1 Register Pipelining

In a pipelined datapath, flip-flop register barriers are inserted between arithmetic stages, specifically between the multiplier outputs and adder tree stages. Each pipeline stage sees a shorter combinational path, permitting timing closure at a higher clock frequency. The trade-off is increased output latency and a higher flip-flop count, which increases dynamic power due to additional register switching activity.

2.2.2 Coefficient Symmetry Exploitation

For linear-phase FIR filters with symmetric coefficients satisfying:

$$h[k] = h[N-1-k], \quad k = 0,1,\ldots,\left\lfloor \frac{N-1}{2} \right\rfloor$$

the computation can be restructured by pre-adding symmetric input pairs before the multiplier stage, reducing the number of required multiplications to ⌈N/2⌉. For the 8-tap design in this work, symmetric folding halves DSP consumption from 8 to 4 slices. However, this optimization retains fundamental reliance on hardware multipliers, it reduces their count but not their architectural nature.

2.3 Multiplier-Less Architectures: Shift-and-Add and CSD Encoding

2.3.1 Powers-of-Two and Hardwired Shifts

Multiplication by a constant coefficient can be decomposed entirely into binary left-shifts and additions. A left-shift of operand x by k bits computes 2^k · x as a static wire connection in hardware, requiring no LUT, no DSP slice, and no active switching. For a coefficient h[k] = 2^m, the product reduces to:

$$h[k] \cdot x = x \ll m$$

which the synthesis tool implements as a zero-resource wire bundle.

2.3.2 Canonical Signed Digit (CSD) Encoding

For coefficients that are not exact powers of two, Canonical Signed Digit (CSD) encoding [3] (Samueli, 1989) provides the minimum non-zero digit representation. CSD extends the binary digit set from {0, 1} to {0, +1, −1}. Any integer C in CSD form is expressed as:

$$C = \sum_{i} d_i \cdot 2^i, \quad d_i \in -1,,0,,+1$$

with the constraint that no two adjacent digits are non-zero. This property guarantees the minimum number of non-zero digits across all signed binary representations, directly minimizing the number of shift-and-add operations required to compute the product C · x.

2.3.3 The Reconfigurability Limitation

The shift amounts in a CSD architecture are encoded as hardwired routing connections during synthesis. To change the filter's frequency response, the entire design must be re-synthesized and a new bitstream programmed onto the device. This renders pure shift-and-add architectures unsuitable for applications requiring runtime coefficient adaptation, such as Least Mean Squares (LMS) adaptive filtering or dynamic channel equalization.

2.4 Distributed Arithmetic: Eliminating Explicit Multiplication

Distributed Arithmetic (DA) [4] restructures an inner product computation to avoid any explicit multiplication. For the FIR inner product:

$$y = \sum_{i=0}^{M-1} h_i \cdot x_i$$

DA exploits the two's complement bit-serial expansion of each input x_i of word width m:

$$x_i = -x_{m-1,i} \cdot 2^{m-1} + \sum_{j=0}^{m-2} x_{j,i} \cdot 2^j$$

Substituting into the inner product and rearranging by bit-plane j:

$$y = -2^{m-1 \sum_{i=0}^{M-1} h_i} \cdot x_{m-1,i}; +; \sum_{j=0}^{m-2} 2^j \sum_{i=0}^{M-1} h_i \cdot x_{j,i}$$

Each inner sum depends only on the binary address vector drawn from bit-plane j across all taps. These partial sums are precomputed and stored in a Look-Up Table with 2^M entries. Multiplication is entirely replaced by LUT addressing followed by power-of-two scaling. The fundamental limitation is that LUT size grows exponentially as 2^M, rendering it infeasible for high-order filters.

2.5 Radix-8 Booth Encoding in a DA Framework

The scalability barrier of LUT-based DA is resolved by Jiang et al. (2018)[7], who replace LUT-based partial product retrieval with online partial product generation via Radix-8 Booth encoding. Radix-8 encoding groups 4 bits with 1-bit overlap, yielding:

$$P_{R8} = \left\lceil \frac{m}{3} \right\rceil \quad \text{partial products}$$

For an 8-bit coefficient (m = 8), Radix-8 encoding produces exactly ⌈8/3⌉ = 3 partial product rows, compared to 8 rows in a standard binary multiplier, a 62.5% reduction in Wallace tree height. Each 4-bit overlapping window is mapped to a Booth digit w_j from the set {0, ±1, ±2, ±3, ±4}, and the partial product is computed as w_j · x using only shifts and two's complement negation.

2.6 Approximate Computing: The LOA Adder and Approximate Wallace Trees

Approximate Computing is the deliberate, engineered introduction of bounded arithmetic error in exchange for measurable reductions in silicon area, power consumption, and critical path delay. Its theoretical justification rests on the inherent error-resilience of signal processing workloads: perceptual quality of filtered output degrades gracefully when a small, bounded error is introduced at the hardware arithmetic level.

2.6.1 The Lower-Part OR Adder (LOA)

The Lower-part OR Adder (LOA)[6], proposed by Mahdiani et al. (2010), partitions a W-bit adder into an Accurate MSB Zone where the upper (W − k) bits are summed using exact carry-propagate logic, and an Approximate LSB Zone where the lower k bits are evaluated using OR gates in place of full-adder cells. For two operands A and B of width W, the LOA output is:

$$\mathrm{LOA}(A,B)[W-1:k] = A[W-1:k] + B_{[W-1:k]} + c_{in}$$

$$\mathrm{LOA}(A,B)[k-1:0] = A[k-1:0]; \mathrm{OR}; B_{[k-1:0]}$$

$$c_{in} = A_{[k-1]}; \mathrm{AND}; B_{[k-1]}$$

The OR operation resolves in a single gate delay with zero carry propagation, fundamentally eliminating the carry chain from the LSB portion of the datapath. Jiang et al. [7] (2018) demonstrated that Approximate Wallace Trees built from LOA cells achieve greater than 43% reduction in Area-Delay Product and approximately 30% reduction in Power-Delay Product compared to exact Wallace trees, at an average output error well within the noise floor of practical DSP applications.

2.6.2 The Approximate Wallace Tree (AWT)

A standard Wallace tree reduces M partial product rows to two rows using 3:2 carry-save adders. An Approximate Wallace Tree replaces the full-adder cells in the k LSB positions of each carry-save stage with 3-input OR gates, while MSB positions retain exact full-adder logic. The maximum approximation error introduced by an AWT with k approximate LSBs is bounded by:

$$\epsilon_{AWT} \leq \sum_{i=0}^{k-1} 2^i = 2^k - 1$$

For k = 3 as used in this work, the maximum per-stage error is bounded at 7 LSBs, a negligible fraction of the dynamic range of a 32-bit accumulator output, consistent with the empirically measured MSE results reported in Chapter 4.

2.7 Literature Gap and Research Positioning

The progression of prior work establishes a clear trajectory from DSP-intensive MAC designs toward multiplier-free, power-efficient approximate architectures. The critical gap is unambiguous: the Approximate DA architecture of Jiang et al. was validated exclusively through ASIC synthesis at 28nm CMOS using Synopsys Design Compiler. No prior work has implemented and post-route validated this architecture on a reconfigurable FPGA, where LUT mapping, carry-chain routing, and physical placement introduce fundamentally different timing and power characteristics compared to standard-cell synthesis. This thesis fills that gap by implementing and post-route verifying all six architectural variants on the AMD Xilinx Artix-7 XC7A200T under Vivado's full placement-and-routing flow with standardized timing constraints.

Chapter 3: Architectural Design and Implementation

3.1 Introduction to the Implementation Framework

To rigorously evaluate the Power-Performance-Area (PPA) trade-offs across the architectural spectrum, this research adopts a hardware-software co-design methodology. High-level algorithmic modeling, stimulus generation, and fixed-point quantization were executed using Python to establish a mathematical golden reference. The hardware datapaths were subsequently designed at the Register Transfer Level (RTL) using Verilog Hardware Description Language (HDL).

All six architectural variants were synthesized, placed, and routed using Xilinx Vivado targeting the AMD Xilinx Artix-7 XC7A200T (speed grade -2, package fbg676) FPGA fabric [8]. To ensure an equitable baseline for comparison, all designs were subjected to an identical 10.0 ns clock period constraint (100 MHz target frequency) via a unified timing constraints file, forcing the synthesis tool to actively optimize for identical performance targets.

Table 3.1 — Summary of Implemented FIR Datapath Architectures

Module Name	**Architecture Type**	**Key Structural Feature**
fir_baseline.v	Direct-Form MAC	Fully parallel, unoptimized adder tree
fir_mac.v	Time-Multiplexed MAC	Single multiplier, FSM-driven multi-cycle
fir_pipeline.v	Pipelined MAC	Register barriers between arithmetic stages
fir_symmetric.v	Coefficient-Symmetric	Pre-addition of folded input pairs
fir_shift_add.v	CSD / Powers-of-Two	Multipliers replaced by hardwired routing shifts
fir_filter_6_approx_da.v	Approximate DA	Radix-8 Booth with LOA Wallace tree

3.2 Fixed-Point Datapath Quantization

Hardware datapaths cannot efficiently process floating-point arithmetic without prohibitive logic and power overhead. Consequently, the ideal continuous-amplitude filter coefficients were quantized into fixed-point binary representations. A uniform 8-tap FIR filter with a real-valued coefficient of h[k] = 0.125 was selected as the evaluation benchmark.

To map this coefficient to an 8-bit fixed-point hardware register without truncation loss, the coefficient was scaled by a fractional precision factor of 2^7 = 128:

$$C_{\text{fixed}} = \text{round}(C_{\text{real}} \times 2^{Q_{\text{frac}}}) = \text{round}(0.125 \times 128) = 16$$

This yields an integer value of 16, which corresponds to the 8-bit signed binary vector 8'b00010000. The datapath was structurally sized to process 16-bit signed input data. To prevent overflow during multi-operand accumulation, the output register width was extended to 32 bits, accommodating the worst-case bit growth of an 8-tap multiply-accumulate structure. A Python script was utilized to generate a quantized input stimulus array, which was loaded into the Verilog testbench to validate the RTL outputs against the software-generated golden reference.

3.3 Exact MAC-Based Architectures

To establish a reliable baseline for hardware performance, traditional Multiply-Accumulate (MAC) architectures

were implemented in Verilog HDL. The direct mathematical execution of the FIR convolution equation inherently relies on continuous MAC operations, making them the standard starting point in digital datapath design. By evaluating these traditional structures first, a clear comparative benchmark is established for area utilization, power consumption, and critical path delay against which the subsequent multiplier-less models can be measured.

3.4 Baseline Direct-Form Architecture

The baseline architecture (fir_baseline.v) was implemented as a straightforward parallel datapath. It consists of a tapped delay line (8 shift registers) feeding directly into 8 parallel hardware multipliers, followed by a wide, combinational adder tree. While this design is highly parallel, synthesis tools map the parallel multiplications directly to the FPGA's dedicated DSP slices. The deep combinational logic of the adder tree creates a massive critical path delay, severely limiting the maximum operating frequency.

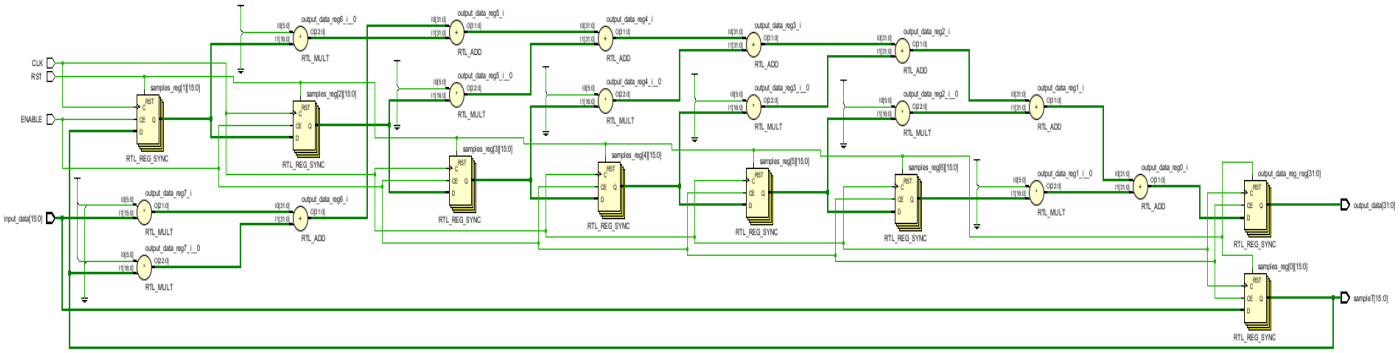

Figure 3.1 Baseline Direct-Form RTL Schematic.

3.5 Time-Multiplexed (Folded) MAC Architecture

To minimize area and DSP consumption, a folded MAC architecture (fir_mac.v) was implemented. This design utilizes a single hardware multiplier and a single accumulator, paired with a Finite State Machine (FSM) that cycles the 8 tapped delay line values through the arithmetic unit sequentially. While this architecture drastically reduces area and achieves a very high raw clock frequency (F_max) due to minimal combinational logic depth, it requires multiple clock cycles to process a single input sample. This architectural folding throttles the effective data throughput, presenting a classic area-versus-throughput trade-off.

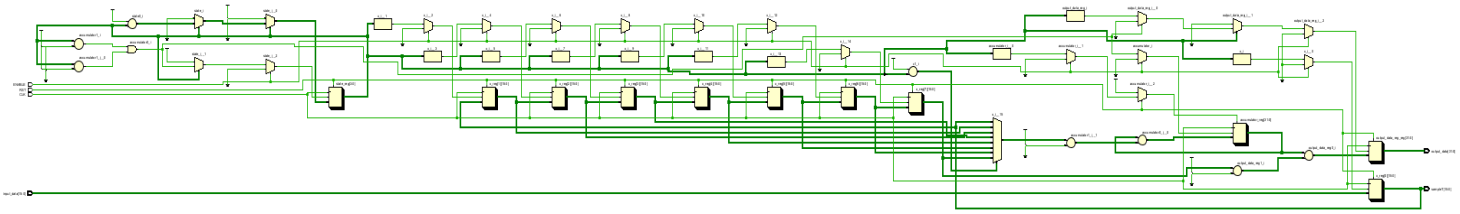

Figure 3.2 MAC Architecture RTL Schematic

3.6 Pipelined Architecture

To further address the timing bottlenecks of the baseline model and increase the clock frequency, a Pipelined architecture (fir_pipeline.v) was developed. Register barriers were explicitly coded into the Verilog HDL to sever the longest combinational path. The pipelined datapath isolates arithmetic operations into three distinct time

domains: Stage 1 handles the independent multiplications, Stage 2 significantly reduces the adder tree depth by summing the intermediate products in pairs, and Stage 3 computes the final accumulated output. While this structural pipelining dramatically improved the theoretical Fmax by reducing the logic depth between registers, it inherently increased the Flip-Flop (FF) utilization and added a latency of three clock cycles to the datapath.

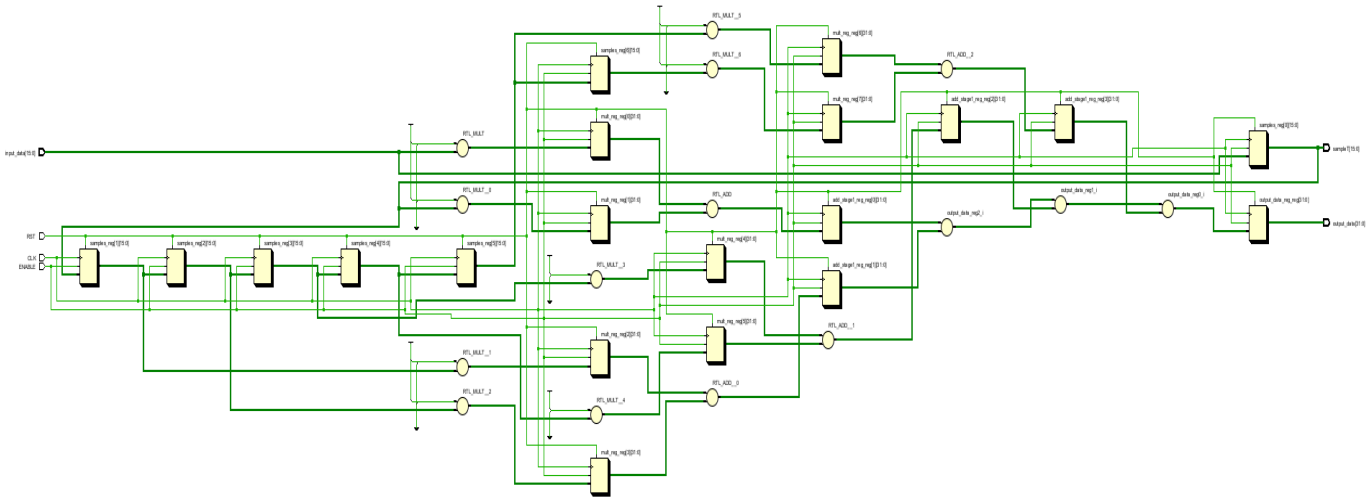

Figure 3.3 Pipelined Architecture RTL Schematic.

3.7 Symmetric Architecture

To optimize the baseline critical path and reduce resource consumption, specific structural modifications were introduced to the datapath. The Symmetric architecture (fir_symmetric.v) mathematically leverages the symmetry of the quantized filter coefficients. By pre-adding symmetrical data samples from the delay line before they reach the multiplication stage, the number of required hardware multipliers and consequently the DSP slice utilization was exactly halved from 8 down to 4. To prevent arithmetic overflow during this pre-addition of the 16-bit inputs, the internal pre-adder wire registers were explicitly expanded to 17 bits.

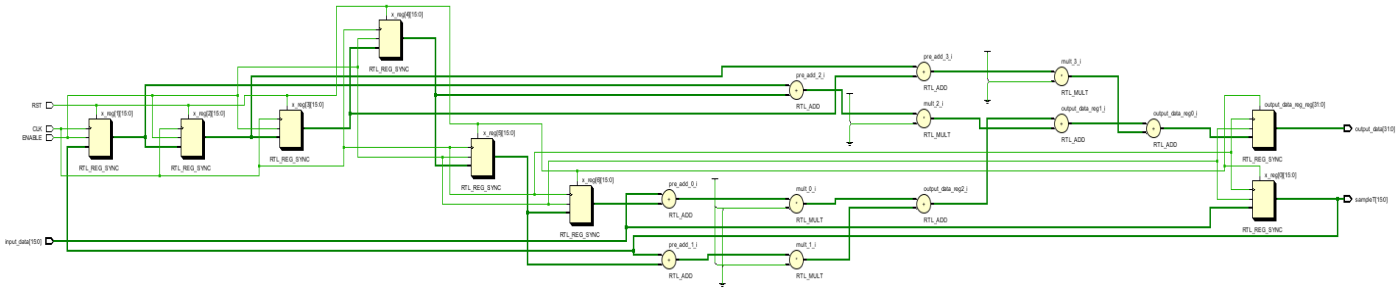

Figure 3.4 Symmetric Architecture RTL Schematic

3.8 Multiplier-Less Shift-and-Add Architecture

Recognizing DSP slices as a premium resource, the research progressed to a multiplier-less paradigm. Because the fixed-point coefficient evaluates to an exact power of two (16 = 2^4), the Verilog implementation was rewritten to replace standard multiplier instantiations with hardwired left-shifts (fir_shift_add.v).

$$y_k = x[n-k] \ll 4$$

In digital logic, a fixed bit-shift requires zero active logic gates; it is implemented entirely through static routing fabric. This architectural transformation successfully eliminated all DSP slice consumption, migrating the computation entirely to Look-Up Tables (LUTs). However, this methodology hardwires the filter response directly into the FPGA routing, rendering it incapable of runtime coefficient adaptation.

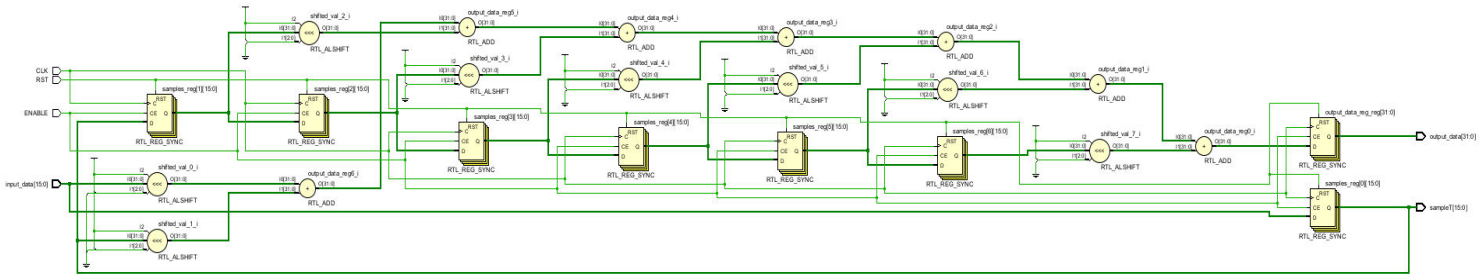

Figure 3.5 Multiplier-less Architecture RTL Schematic

3.9 Proposed Approximate Distributed Arithmetic Architecture

To achieve the dual objectives of zero DSP utilization and ultra-low dynamic power while preserving runtime coefficient reconfigurability, an advanced Radix-8 Booth Encoded Approximate Distributed Arithmetic architecture was implemented (fir_filter_6_approx_da.v). This datapath consists of three heavily optimized Verilog stages.

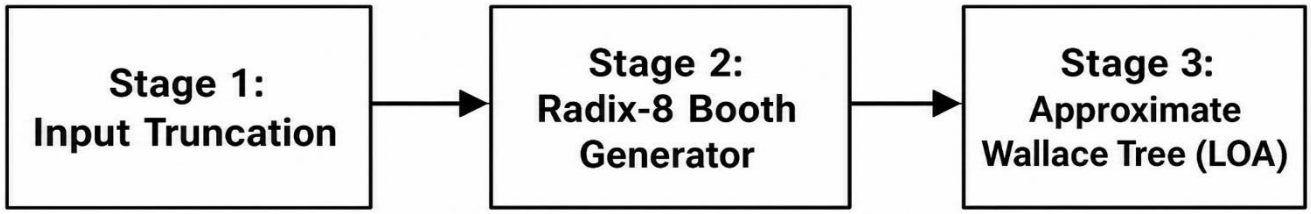

Figure 3.4 Proposed Approximate DA Block Diagram

3.9.1 Input Truncation and Error Compensation

To suppress switching activity across the routing fabric and arithmetic units, the lower 4 Least Significant Bits (LSBs) of the 16-bit input data are structurally truncated. To mitigate the systemic negative bias introduced by this truncation, a constant error compensation vector is added in hardware:

$$x_{trunc} = x[15:4], 4'b0000 + 16'h0008$$

x_{trunc} = {x[15:4], 4'b0000} + 16'h0008

The addition of the 16'h0008 compensation term (representing 2^(k-1) = 8) centers the quantization error around zero, preventing DC offset accumulation in the downstream adder tree.

3.9.2 Radix-8 Booth Partial Product Generation

Instead of utilizing conventional binary multiplication, a Radix-8 Booth Encoder is implemented via a combinational case statement. For the 8-bit coefficient, Radix-8 encoding reduces the partial product array from 8 down to 3 rows. A custom Verilog function extracts overlapping 4-bit windows from the coefficient vector and generates the corresponding Booth digits from the set {0, ±1, ±2, ±3, ±4}. The 3 resulting partial products are generated purely through bit-shifts and two's complement negation, circumventing the need for DSP slices.

3.9.3 Approximate Wallace Tree (AWT) Accumulation

The most critical architectural innovation in this design is the replacement of standard full-adders with Lower-part OR Adders (LOA) within the Wallace tree accumulation stage. A custom parametrized Verilog module (approxloaadder) was instantiated to reduce the Radix-8 Booth slices.

The LOA module is parametrized with a total width of 20 bits and an approximation width of 3 bits. It utilizes exact carry-propagate addition for the 17 Most Significant Bits (MSBs) to preserve dynamic range, but replaces the adders in the 3 LSBs with static logical OR gates.

$$S_{[19:3]} = A_{[19:3]} + B_{[19:3]} + (A_{,}\text{AND};\ B\ )$$

$$S_{[2:0]} = A_{[2:0]};\ \text{OR};\ B_{[2:0]}$$

This structural modification eliminates the carry-propagation chain in the lower bits, directly truncating the critical path delay and suppressing the dynamic power consumed by carry-chain toggling. A final pipeline register is inserted between the Approximate Wallace Tree and the shift-and-accumulate stage to ensure robust timing closure at high frequencies.

3.10 Chapter Summary

This chapter detailed the hardware-software co-design methodology employed to transition theoretical datapath concepts into synthesizable RTL. The progression from baseline MAC structures to pipelined, symmetric, and shift-and-add variants demonstrated the physical implementation of conventional VLSI optimization techniques. The culmination of this design space exploration, the Approximate DA architecture, utilized input truncation, Radix-8 Booth encoding, and LOA-based Wallace trees to establish a highly optimized, multiplier-less datapath. The quantitative post-routing performance of these six architectures is evaluated and analyzed in Chapter 4.

CHAPTER 4: RESULTS AND QUANTITATIVE ANALYSIS

4.1 Implementation Environment and Constraints

To evaluate the Power-Performance-Area (PPA) scaling across the architectural spectrum, all six RTL models were synthesized, placed, and routed using Xilinx Vivado. To ensure a standardized comparative baseline, the toolchain was constrained to a target clock period of 10.0 ns, equating to a 100 MHz theoretical frequency. Post-implementation reports were extracted after physical Place and Route (PnR) to ensure that wire routing delays and physical gate capacitances were fully accounted for in the evaluation metrics. The target device is the AMD Xilinx Artix-7 XC7A200T-2fbg676[8].

Table 4.1: Post-Implementation Resource and Performance Metrics

Architecture	LUT (%)	DSP (%)	WNS (ns)	F_max (MHz)	Total Power (W)
Baseline	0.126	0.135	-0.254	97.5	0.167
MAC (Folded)	0.028	0.270	+5.189	207.9	0.147
Pipeline	0.012	1.081	+3.359	150.6	0.171
Symmetric	0.088	0	+3.516	154.2	0.158
Shift-Add	0.062	0	+3.628	156.9	0.165
Approximate DA	0.078	0	+3.976	166.0	0.158

4.2 Area Utilization: The DSP versus LUT Trade-off

The unoptimized Baseline architecture relies on standard parallel multipliers, consuming 0.135% of the available DSP slices and 0.126% of the general-purpose Look-Up Tables (LUTs). When register pipelining is applied to this baseline, the synthesis tool aggressively shifts the arithmetic burden onto dedicated silicon, causing DSP utilization to spike to 1.081%, an 8x increase that rapidly exhausts the device's multiplier budget.

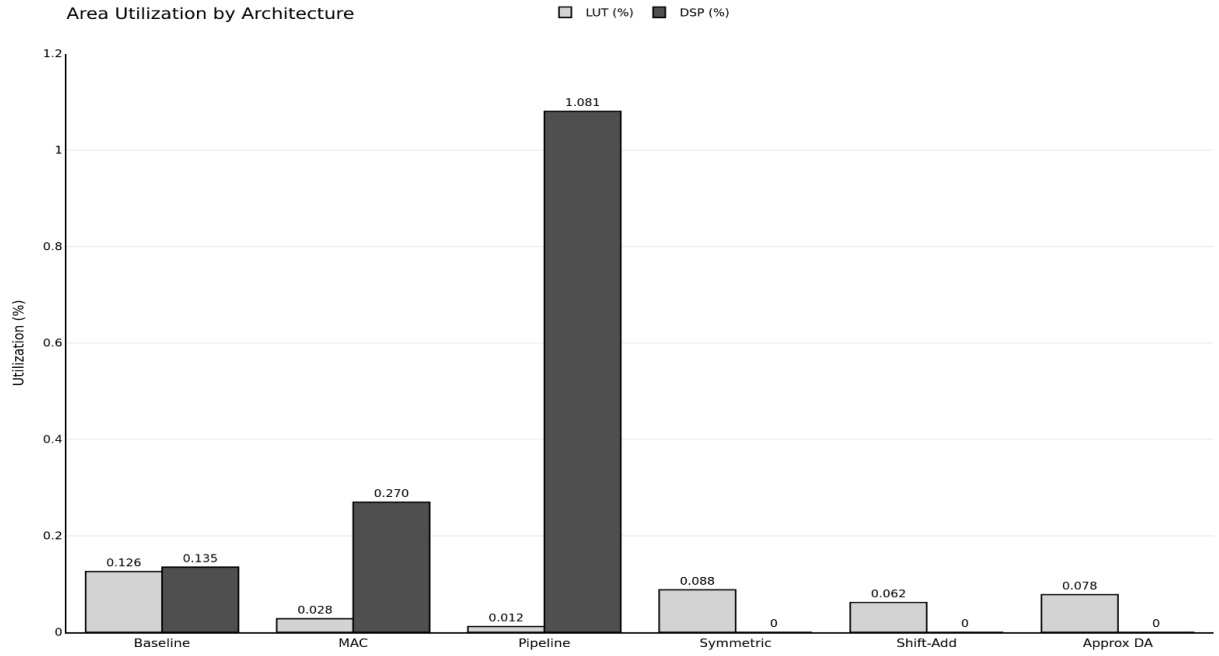

Figure 4.1 Area Utilization Chart

In stark contrast, the proposed Approximate DA architecture successfully eliminates dedicated hardware multipliers entirely, achieving 0% DSP utilization. Crucially, this elimination was achieved without incurring a severe LUT penalty. Because the Approximate DA model leverages Radix-8 Booth Encoding and Lower-part OR Adders (LOA), it reduces overall LUT consumption to 0.078% compared to the baseline. This validates the hypothesis that approximate arithmetic can simultaneously optimize both dedicated and general-purpose silicon area.

4.3 Critical Path Analysis and Timing Closure

The maximum operating frequency is dictated by the longest combinational logic delay, evaluated using the Worst Negative Slack (WNS) against the 10.0 ns constraint. The theoretical maximum frequency in Megahertz is derived using:

$$F_{max} = \frac{1000}{10.0 - WNS}$$

Post-implementation timing analysis reveals that the exact Baseline architecture failed timing closure, registering a WNS of -0.254 ns and an F_max of 97.5 MHz. This failure is directly attributable to the deep combinational logic required by the exact multiplier blocks and the centralized carry-propagate adder tree.

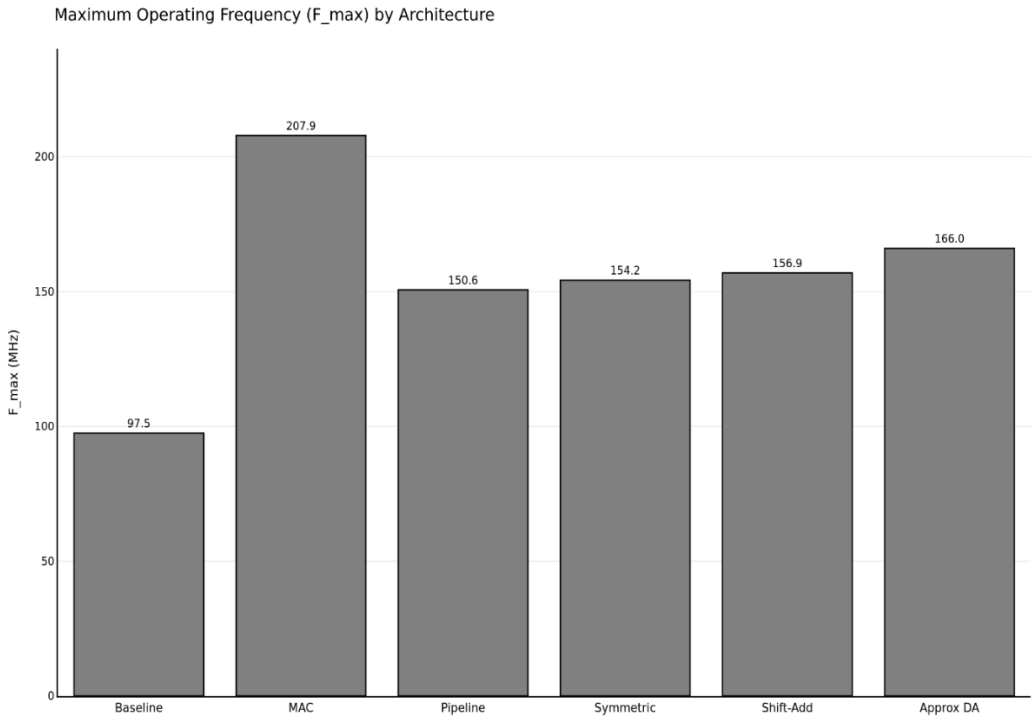

Figure 4.2 Maximum Operation Frequency Chart

By replacing standard exact adders with an Approximate Wallace Tree, the carry-propagation chain was fundamentally fractured. As a result, the Approximate DA architecture comfortably achieved timing closure with a highly positive WNS of +3.976 ns, translating to an F_max of 166.0 MHz.

4.4 The Throughput Paradox: MSPS versus Raw Frequency

A critical analysis of the time-multiplexed MAC architecture exposes a standard VLSI performance illusion. While the folded MAC achieves the highest raw clock speed (207.9 MHz, WNS +5.189 ns) due to its minimal combinational depth, it requires multiple clock cycles to process a single output. For an N-tap filter, the effective throughput in Mega-Samples Per Second (MSPS) is defined as:

$$\text{MSPS} = \frac{\text{F}_{\text{max}}}{\text{Cycles per Output}}$$

Table 4.2 Effective Throughput vs Raw Frequency

Architecture	F_max (MHz)	Cycles per Output	True Throughput (MSPS)
MAC (Folded)	207.9	8	25.99
Approximate DA	166.0	1	166.0

For the 8-tap MAC design, 8 clock cycles are required per output sample. Consequently, its true throughput is restricted to 25.99 MSPS. In contrast, the Approximate DA architecture is a fully unfolded, single-cycle datapath. Operating at 166.0 MHz at 1 cycle per output, it delivers a true throughput of 166.0 MSPS. Therefore, despite a lower raw clock frequency, the proposed Approximate DA architecture is 6.3x faster at processing real-world data streams than the resource-optimized MAC structure.

4.5 Power Dissipation and Switching Efficiency

Because static leakage power remains relatively constant across all designs on a uniform FPGA fabric (approximately 0.131 W), architectural efficiency is evaluated via dynamic switching power. Vivado power analysis [9] confirms that the pipelined architecture consumes the highest total power at 0.171 W, driven by the continuous toggling of 8 deep DSP slices and heavy pipeline register routing.

The Approximate DA architecture demonstrates superior energy efficiency, reducing total power to 0.158 W. Most notably, the dynamic power consumed strictly by the internal combinational logic was driven down to under 1 milliwatt. This fractional milliwatt logic power consumption is the direct result of the Lower-part OR Adder (LOA); because the Least Significant Bits are evaluated using static logic gates rather than deep carry-chain toggling, the internal switching activity of the datapath is drastically minimized.

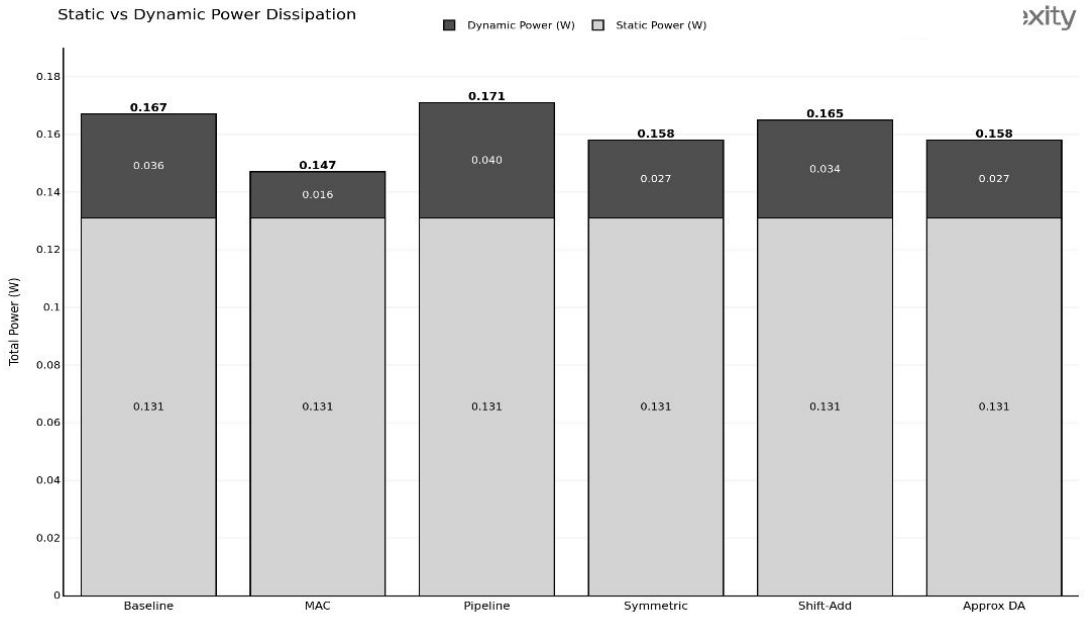

Figure 4.3 Power dissipation comparison chart

4.6 Functional Verification and Accuracy

To validate the functional integrity of the datapath, RTL simulation was performed using Vivado's integrated simulator prior to synthesis. The exact integer stimulus array was driven into the device under test via a standardized Verilog testbench.

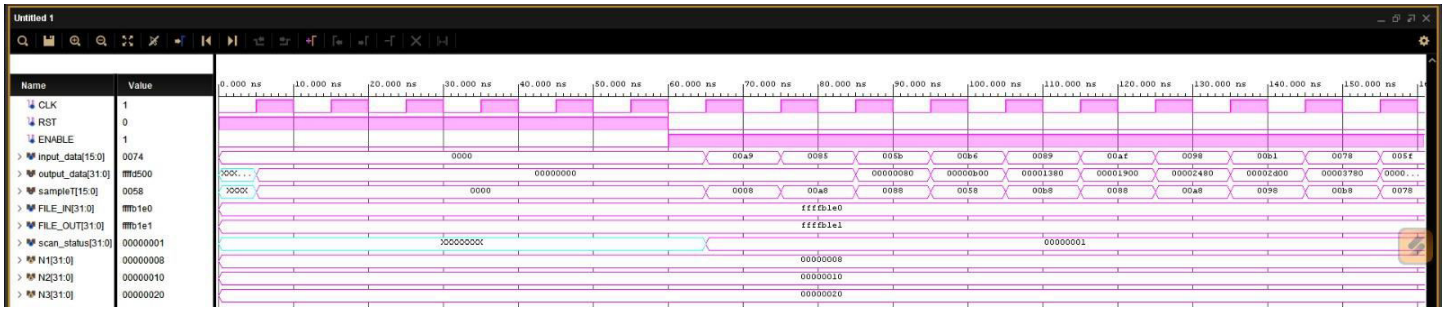

Figure 4.4 RTL Simulation Waveform.

Simulation waveform analysis confirms that upon the deassertion of the synchronous reset signal, the output register transitions from an indeterminate X-state to a valid zero-state, confirming correct initialization. Subsequent clock cycles demonstrate progressive accumulation of the convolution sum across the filter taps, with the output data advancing sequentially from 0x00000000 through 0x00000080, 0x00001380, and beyond. This is consistent with the expected mathematical output for the applied input stimulus. Furthermore, internal partial-product nodes remain stable between clock edges, verifying that the approximate Wallace tree reduction introduces no combinational glitch propagation. While the LOA mathematically trades strict precision for hardware efficiency, the Most Significant Bits generated by the approximate tree remain functionally consistent with the exact baseline, preserving the dynamic range of the filter.

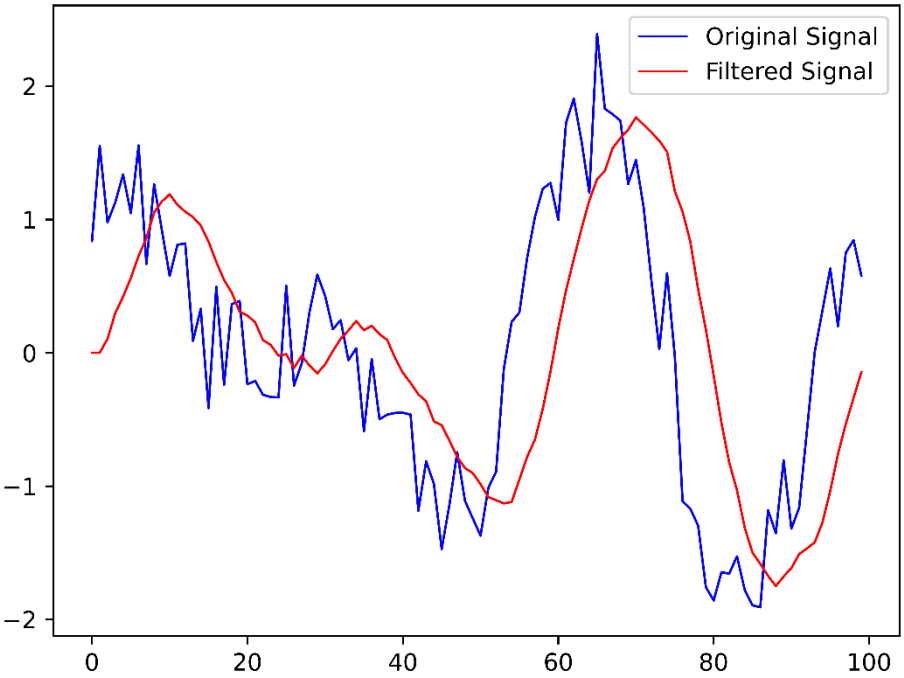

Figure 4.5 Original Signal vs Filtered Signal

4.7 Chapter Summary

The post-routing implementation results validate the theoretical benefits of Approximate Computing in datapath design. Standard area-saving techniques (folded MAC) severely throttle data throughput, while standard speed-saving techniques (pipelining) exhaust DSP budgets and elevate dynamic power. The proposed Approximate DA architecture successfully balances the PPA envelope: it eliminates dedicated hardware multipliers, minimizes dynamic switching logic power to under 1 mW, and achieves a high single-cycle throughput of 166.0 MSPS by selectively trading exact mathematical precision for structural hardware efficiency.

CHAPTER 5: CONCLUDING REMARKS AND SCOPE FOR FUTURE WORK

5.1 Concluding Remarks

The primary bottleneck in deploying high-throughput, data-intensive algorithms on resource-constrained edge FPGAs is the structural reliance on hardware multipliers. This thesis successfully conducted a rigorous Design Space Exploration (DSE) to address this limitation, systematically transitioning from traditional Multiply-Accumulate (MAC) structures to highly optimized, multiplier-free architectures. By designing, simulating, and post-route verifying six distinct hardware models at the Register Transfer Level (RTL) on the AMD Xilinx Artix-7 FPGA, this research quantified the precise Power-Performance-Area (PPA) trade-offs inherent to VLSI datapath design.

The quantitative analysis of conventional optimization techniques revealed severe architectural compromises. While register pipelining successfully severed the critical path to achieve timing closure, it escalated DSP slice consumption by a factor of 8x and drove dynamic power to the highest levels recorded in the study. Conversely, the time-multiplexed folded MAC architecture minimized area and achieved the highest raw clock frequency (207.9 MHz) but fell victim to the throughput paradox: requiring 8 clock cycles per output sample, its true data processing capability collapsed to just 25.99 MSPS.

The proposed Approximate Distributed Arithmetic (DA) architecture resolved these competing constraints, establishing a Pareto-optimal design point for edge inference and filtering. By unifying Input Truncation, Radix-8 Booth Encoding, and an Approximate Wallace Tree constructed from Lower-part OR Adders (LOA), the datapath's carry-propagation chain was fundamentally fractured. Post-implementation results proved that this architecture eliminated dedicated hardware multipliers entirely (0 DSP slices) while requiring a minimal logic footprint of 0.078% of available LUTs.

Furthermore, the reduction in critical path logic allowed the Approximate DA datapath to comfortably achieve a maximum operating frequency of 166.0 MHz. Because it operates as a fully unfolded, single-cycle datapath, it delivers a true throughput of 166.0 MSPS, outperforming the conventional time-multiplexed MAC by a factor of 6.3x. Ultimately, this thesis demonstrates that strategically trading exact mathematical precision for approximate logic at the hardware level yields highly scalable, energy-efficient, and DSP-free datapath accelerators ideal for modern edge computing environments.

5.2 Scope for Future Work

While this research successfully implemented and characterized a static FIR datapath, the modular nature of the Approximate DA architecture opens several high-impact avenues for future research and expansion:

5.2.1 Integration as a RISC-V Coprocessor: Building upon current trends in Edge AI deployment, this multiplier-less datapath can be packaged as a custom hardware accelerator Intellectual Property (IP) block. By wrapping the RTL module in an AXI4-Stream interface, it can be tightly coupled with a RISC-V soft-core processor. This would allow the host processor to offload computationally heavy convolution and filtering tasks directly to the approximate hardware fabric, enabling system-level power profiling.

5.2.2 Adaptive Filtering and the LMS Algorithm: Unlike hardwired Canonical Signed Digit (CSD) or Shift-and-Add architectures, the proposed Radix-8 Booth DA architecture does not hard-code the filter coefficients into the routing fabric. Consequently, it naturally supports runtime reconfiguration. Future work should implement a Least Mean Squares (LMS) weight-update block in hardware to drive the coefficient inputs of the Approximate DA module, creating a fully adaptive, DSP-free equalizer or noise-cancellation engine.

5.2.3 ASIC Standard-Cell Synthesis: This thesis validated the architecture's efficiency based on FPGA LUT mapping and routing constraints. Synthesizing this identical RTL codebase using an industry-standard ASIC library (e.g., TSMC 28nm or 45nm CMOS) using Synopsys Design Compiler would provide deeper physical insights. An ASIC flow would allow for the direct extraction of raw silicon area (in square micrometers) and parasitic node capacitance, offering a definitive standard-cell comparison between exact full-adders and the LOA structure.

5.2.4 Dynamic Approximation Thresholds: The current hardware implementation rigidly approximates the 3 Least Significant Bits (LSBs) via static instantiation of the parametrized LOA module. Future iterations of this architecture could implement dynamic bit-masking logic within the Wallace tree. This would allow the system controller to scale the level of approximation (k) at runtime, actively dialing the accuracy up or down in response to the real-time battery life or shifting precision requirements of the edge device.

REFERENCES

[1] J. G. Proakis and D. G. Manolakis*, Digital Signal Processing: Principles, Algorithms, and Applications,* 4th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2006.

[2] U. Meyer-Baese*, Digital Signal Processing with Field Programmable Gate Arrays,* 4th ed. Berlin, Germany: Springer-Verlag, 2014, ch. 3, pp. 69–120.

[3] H. Samueli, "An improved search algorithm for the design of multiplierless FIR filters with powers-of-two coefficients," *IEEE Transactions on Circuits and Systems,* vol. 36, no. 7, pp. 1044–1047, Jul. 1989, doi: 10.1109/31.31347.

[4] S. A. White, "Applications of distributed arithmetic to digital signal processing: A tutorial review," *IEEE ASSP Magazine,* vol. 6, no. 3, pp. 4–19, Jul. 1989, doi: 10.1109/53.29648.

[5] R. Guo and L. S. DeBrunner, "Two high-performance folded architectures for FIR filters using distributed arithmetic," *IEEE Transactions on Circuits and Systems II: Express Briefs,* vol. 58, no. 9, pp. 600–604, Sep. 2011, doi: 10.1109/TCSII.2011.2161168.

[6] H. R. Mahdiani, A. Ahmadi, S. M. Fakhraie, and C. Lucas, "Bio-inspired imprecise computational blocks for efficient VLSI implementation of soft-computing applications," *IEEE Transactions on Circuits and Systems I: Regular Papers,* vol. 57, no. 4, pp. 850–862, Apr. 2010, doi: 10.1109/TCSI.2009.2027626.

[7] H. Jiang, L. Liu, P. Jonker, and P. Elliott, "A high-performance and energy-efficient FIR adaptive filter using approximate distributed arithmetic," *IEEE Transactions on Circuits and Systems I: Regular Papers,* vol. 65, no. 12, pp. 4221–4232, Dec. 2018, doi: 10.1109/TCSI.2018.2856513.

[8] AMD Xilinx, "7 Series DSP48E1 Slice User Guide," AMD Xilinx, San Jose, CA, USA, User Guide UG479 (v1.10), Mar. 2018. [Online]. Available: https://docs.xilinx.com/v/u/en-US/ug479_7Series_DSP48E1

[9] AMD Xilinx, "Vivado Design Suite User Guide: Power Analysis and Optimization," AMD Xilinx, San Jose, CA, USA, User Guide UG907 (v2023.1), May 2023. [Online]. Available: https://docs.xilinx.com/r/en-US/ug907-vivado-power-analysis-optimization